Consumers are triggered by a message arriving in the queue. RabbitMQ provides an *at-least-one delivered* guarantee, so you need to be aware of possible duplicates. The code we implemented updates the status of the specific order to be `DISPATCHED`, an operation that can be executed several times with the same result. Since the operation is idempotent, the code is resilient to duplicates. A further optimization would be to check for the status and skip the update operation if it's already dispatched.

Finally, we need to configure Spring Cloud Stream in the application.yml file so that the `dispatchOrder-in-0` binding (inferred from the `dispatchOrder` function name) is mapped to the `order-dispatched` exchange in RabbitMQ. Also, remember to define `dispatchOrder` as the function that Spring Cloud Function should manage, and the integration with RabbitMQ.

Listing 10.21 Configuring Cloud Stream bindings and RabbitMQ integration

```
spring:
  cloud:
    function:
      definition: dispatchOrder                ❶
    stream:
      bindings:
        dispatchOrder-in-0:                    ❷
          destination: order-dispatched        ❸
          group: {spring.application.name}     ❹
  rabbitmq:                                    ❺
    host: localhost
    port: 5672
    username: user
    password: password
    connection-timeout: 5s
```

❶ Definition of the function managed by Spring Cloud Function

❷ The input binding

❸ The actual name at the broker that the binder binds to (the exchange in RabbitMQ)